SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1441 CSA14 **ME: Compiler Design for Engineers**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: **Total Marks:50**

Mode: Take-Home **Hours : 1 Day**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Challa Priyadarshini(192311118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Challa Priyadarshini*

Q. No	Error Identification (20)	Root Cause Analysis (30)	Correction & Justification (25)	Application of Concepts (15)	Clarity & Presentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 30	/ 25	/ 15	/ 10	/ 100

Annexure A

ERROR ANALYSIS TASK

Q1. Full Compiler Phase Error Analysis (BTL4) (10 Marks)

The following C program is intended to perform arithmetic operations, conditional checks, and looping constructs. However, it contains multiple errors across different phases of compilation including lexical, syntax, semantic, logical, and runtime errors.

```
#include <stdio.h>

int main() {
int num1 = 25;
float x = 3.5
int y;

y = num1 + 10

if(y == 35 {
printf("Correct Value\n")
}

int arr[3] = {1,2,3};
printf("%d\n", arr[3]);

int *p;
*p = 20;

int z = 10 / 0;

while(num1 < 30) {
printf("%d\n", num1);
num1++;
}

eturn 0;
```

asks:

) Identify all errors with description
) Classify errors into lexical, syntax, semantic, logical, and runtime errors
) Explain which errors are detected during compilation and which occur at runtime

(d) Rewrite the corrected version of the program
(e) Analyze the impact of division by zero and pointer misuse
(f) Suggest improvements in compiler design for better error detection

Q2. Advanced Lexical Error Identification (BTL4) (10 Marks)

The following C program contains multiple lexical errors due to invalid identifiers, malformed constants, and incorrect symbols. Carefully analyze the program and answer the questions.

```
#include <stdio.h>

int main() {
int 5num = 10;
float value& = 2.5;
char ch = 'a';
int total = 10#20;
int _data = 15;
int data@1 = 25;
printf("Sample Program);
int y = 0xH12;
float f = 3.4.5;
char str[] = "Hello;
return 0;
}
```

Tasks:

(a) Identify all lexical errors with invalid tokens
(b) Explain which lexical rule is violated (identifier, constant, literal, symbol)
(c) Rewrite each invalid token into a valid token
(d) Explain how a lexical analyzer processes such errors
(e) Design a DFA rule to detect invalid identifiers starting with digits
(f) Suggest improvements in lexical error reporting

Q3. Intermediate Code and Optimization Analysis (BTL4) (10 Marks)

The following C program demonstrates arithmetic computation and optimization concepts but contains inefficiencies and errors.

```
#include <stdio.h>

int main() {
int a = 5;
int b = 10;
int c;

c = a * b + a * b + b * a;

if(a = 5) {
printf("Check\n");
}

int arr[2] = {1,2};
printf("%d\n", arr[2]);

int *ptr;
*ptr = 10;

return 0;
}
```

Tasks:

(a) Identify all errors and inefficiencies in the program
(b) Classify errors according to compiler phases
(c) Generate three-address code (TAC) for the expression
(d) Apply DAG representation to eliminate redundant computations
(e) Generate optimized target code with minimal register usage
(f) Analyze the runtime impact of array bounds violation and pointer misuse

Q4. Error Identification and Classification (BTL4) (10 Marks)

Analyze the following C program.

```
#include <stdio.h>

int main() {
   int x = 10
   int y = 20;
   int z = x + y;
   printf("%d\n", z);
   return 0;
}
```

Tasks:

(a) Identify all errors in the program.
(b) Classify each error as lexical, syntax, semantic, logical, or runtime.
(c) Rewrite the corrected program.

Q5. Compiler Error Analysis (BTL4) (10 Marks)

Analyze the following C program.

```
#include <stdio.h>

int main() {
    int arr[2] = {5,10};
    printf("%d\n", arr[3]);

    int *ptr;
    *ptr = 50;

    return 0;
}
```

Tasks:

(a) Identify the runtime errors in the program.
(b) Explain why these errors occur.
(c) Rewrite the corrected version of the program.

Bottom of Form

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors

Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

CO1 - AT - 2 Error Analysis task

1. Full compiler phase Error Analysis

a) Errors identification

from given code;

1. Missing ; after float x = 3.5
2. Missing ; after y = num1 + 10
3. Missing ; in if (y == 35)
4. Missing ; after printf()
5. Accessing arr[3] (array out of bounds)
6. uninitialized pointer *p = 30
7. Division by zero 10/0.

b) classification of errors

- Syntax : Missing semicolons, missing)
- Runtime : Array index out of bounds, pointer misuse, division by zero.
- Lexical : None
- Semantic : None
- Logical : None

c) Compile-time Errors

- Missing ;
- Missing ;

Runtime errors

- Array out of bounds
- pointer misuse.
- Division by zero.

D) ~~correct~~ corrected program

```
#include<stdio.h>
int main(){
    int num1 = 25;
    float x = 3.5;
    int y;
    y = num1 + 10;
```

```
if(y==35){
    printf("correct value\n"); }
int array[3] = {1,2,3};
printf("%d\n", array[2]);
   int value = 20;
   int *p = &value;
     *p = 20;
     while (num1 < 30){
       printf("%d\n", num1);
           num1++; }
         return 0;
         }
```

e) Impact of division by zero and pointer misuse.

- Division by zero → program crash / undefined behavior
- pointer misuse → segmentation fault or each.

f) Improvements in complier design

- Better syntax checking
- Array bounds checking
- pointer analysis
- clear error messages

Advanced lexical Error identification

a) Invalid tokens

- $num
- value &
- 10#20
- data@1
- sample program
- 0x1H2
- 3.4.5
- "Hello

b) violated rules(identifier, constant, literal, symbol)

→ Identifier starts with digit.

→ Invalid symbol

→ Invalid operator

→ Invalid hexadecimal constant
→ Invalid floating constant
→ unterminated string literal.

c) connections

- 5num → nums (should not start with num)
- value2 → value
- 10#20 → 10+20
- data@1 → data1
- "sample program' (close quote)
- 0x12
- 3.45
- "Hello"

d) Lexical analyzer

Reads source code, forms, tokens, reports, invalid tokens, then passes valid tokens to parser.

e) DFA Rule to detect invalid identifiers.

First character → letter or Remaining characters → letter/Digit/-

f) Improvements in lexical error reporting.

* Display line number
* Highlight invalid token
* Give correction suggestion

3. Intermediate code & optimization

a. Errors Identification in the program

- Repeated computation a * b
- if (a=5) uses assignment
- arr[a] → out of bounds
- uninitialized pointer.

b. classification of errors

- Syntax : None
- Semantic : Assignment in condition
- Runtime : Array bounds, pointer misuse

- optimization : Repeated expression.

c. Three-address code (TAC) for the expression

$t_1 = a * b$

$t_2 = t_1 + t_1$

$t_3 = t_2 + t_1$

$c = t_3$

d. DAG representation

compute a * b once and reuse it.

e. optimized Target code

$R_1 = a * b$

$R_2 = R_1 + R_1$

$R_2 = R_2 + R_1$

$c = R_2$

f. Runtime Impact

- Array violation → undefined behaviour.
- pointer misuse → crash.

Error Identification & classification

a) Error identification

- Missing ; after int x=10

b) classification

- syntax Error

c) corrected program

```
# include <stdio.h>
int main(){
int x=10;
int y=20;
int z=x+y;
printf("%d\n", z);
return 0;
}
```

compiler Error analysis

a) Runtime Errors

- arr[3] (array out of bounds)

- *ptr = 50 (uninitialized pointer)

b) Reason why errors occurs.

- Invalid array index accesses memory outside array.
- pointer doesn't point to valid memory

c) corrected program

```
#include <stdio.h>
int main(){
  int arr[2]: {5,10};
 printf("%.d\n", arr[i]);
  int value = 50;
   int *ptr = &value;
 printf("%.d\n", *ptr);
     return 0;
      }
```